Модуль 5

fwdays academy

LangChain & LangGraph: Розробка AI Агентів

Що таке LangChain?

Open-source фреймворк для створення додатків на основі великих мовних моделей (LLM)

Ключові можливості:

- Інтеграція з різними LLM провайдерами
- Керування промптами та їх оптимізація
- Створення складних ланцюгів обробки
- Робота з векторними базами даних
- Інтеграція з зовнішніми API та інструментами

Від базових chains до мультиагентних систем

- LangChain Official Site
- LangGraph Documentation
- GitHub Stars: 110k+

Факти про екосистему:

- **70+ млн завантажень/місяць** (більше ніж OpenAI SDK)
- **700+ інтеграцій** з LLMs, векторними БД, tools
- **Enterprise adoption:** Uber, LinkedIn, Klarna, JP Morgan
- **$1.1B оцінка** (Series B, 2025)

LangChain Platform GA Announcement

Top 5 LangGraph Agents in Production 2024

Еволюція версій

Timeline змін

v0.1 (Жов 2022) → v0.2 (Тра 2024) → v1.0 (Жов 2025)

Ключові важливі зміни:

- ❌ AgentExecutor → ✅ create_agent
- ❌ LLMChain → ✅ LCEL (LangChain Expression Language: prompt | model | parser)
- ❌ ConversationBufferMemory → ✅ Checkpointers

Посилання:

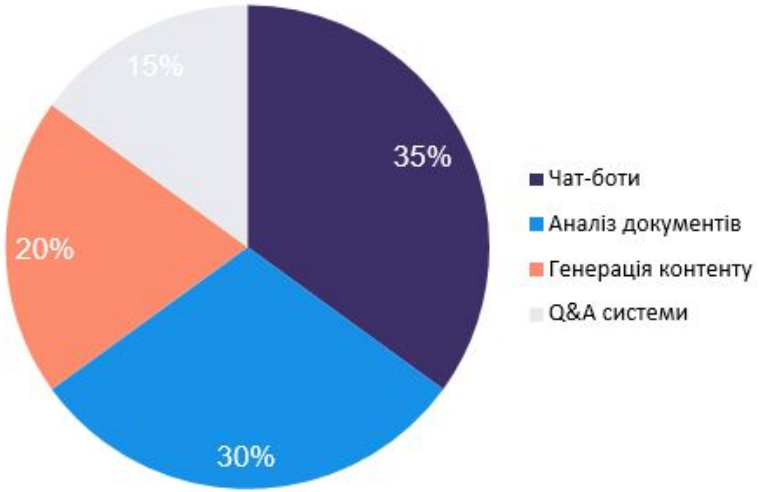
- LangChain v1.0 Release Notes
- Migration Guide
- Історія релізів

Популярні застосування

Основні сценарії:

- Чат-боти та асистенти
- Аналіз документів
- Генерація контенту
- Q&A системи

Популярність рішень:



Модульна структура

```
langchain-core (базові абстракції)
    ├── langchain (chains, agents)
    ├── langchain-community (інтеграції)
    └── partner packages
        ├── langchain-openai
        ├── langchain-anthropic
        └── langchain-google
```

langchain-core - базовий пакет з абстракціями та інтерфейсами:

- Базові інтерфейси (Runnable, BaseLLM, BaseRetriever)
- Протоколи для промптів, виводу, callbacks
- Основні типи даних та схеми
- LCEL (LangChain Expression Language)

langchain - основний пакет-агрегатор:

- Chains (готові ланцюжки)
- Agents (автономні агенти)
- Memory (системи пам'яті)
- Callbacks та utilities
- Залежить від core та community

langchain-community - інтеграції зі сторонніми сервісами:

- Різні провайдери LLM (не-основні)
- Векторні бази даних
- Document loaders
- Embedding моделі
- Tools та utilities від спільноти

LCEL (LangChain Expression Language)

```
# Старий підхід (deprecated!)
chain = LLMChain(llm=model, prompt=prompt)
result = chain.run(topic="AI")

# Новий підхід (використовуйте це!)
chain = prompt | model | StrOutputParser()
result = chain.invoke({"topic": "AI"})
```

Переваги: Streaming by default, Async support, Parallel execution, LangSmith tracing

- LCEL Tutorial

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import
StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "You are an expert in {domain}. Answer: {question}"
)
model = ChatOpenAI(model="gpt-4")
parser = StrOutputParser()

chain = prompt | model | parser

# RunnableParallel для швидкості
from langchain_core.runnables import RunnableParallel

parallel_chain = RunnableParallel({
        "summary": summary_chain,
        "sentiment": sentiment_chain,
        "keywords": keyword_chain })

results = parallel_chain.invoke({"text": document})
```

create_agent

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

# Новий спрощений API (v1.0)
agent = create_agent(
    model=ChatOpenAI(model="gpt-4"),
    tools=[web_search, calculator, database],
    system_message="You are a helpful assistant"
)

# Відсутній AgentExecutor
result = await agent.ainvoke({"input": "What's the weather?"})
```

LC Agent documentation

Tools в langchain

Компоненти, які агенти викликають для виконання дій

- Інкапсулюють виконувану функцію та її вхідну схему
- Передаються сумісним chat моделям
- Модель вирішує, чи викликати tool і з якими аргументами
- Розширюють можливості моделей для взаємодії з API, базами даних, файловими системами

Tools

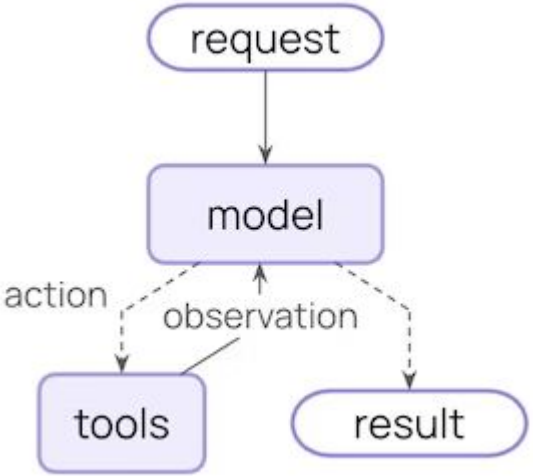
```python
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the customer database for records.

    Args:
        query: Search terms to look for
        limit: Maximum number of results to return
    """
    return f"Found {limit} results for '{query}'"


# Type hints обов'язкові — визначають вхідну схему
# Docstring стає описом для моделі
```

ToolRuntime

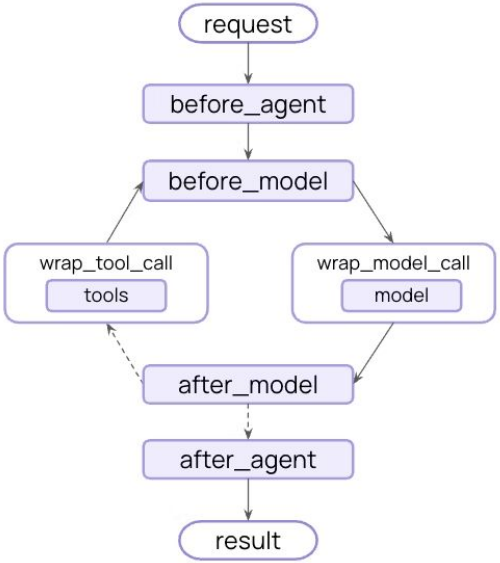
Доступ до контексту виконання

- State (Стан) — Змінні дані (повідомлення, лічильники, кастомні поля)
- Context (Контекст) — Незмінна конфігурація (user ID, session деталі)
- Store (Сховище) — Постійна довгострокова пам'ять між розмовами
- Stream Writer — Потокова передача кастомних оновлень

Middleware



Цикл основного агента включає виклик моделі, вибір інструментів для виконання та завершення, коли виклики інструментів припиняються



Middleware надає доступ до перехоплювачів до та після кожного з цих кроків

Модуль 5 .2

fwdays academy

LangGraph

LangGraph

LangGraph — це низькорівнева оркестраційна платформа та середовище виконання для створення, керування та розгортання тривало працюючих агентів із збереженням стану, якій довіряють компанії, що формують майбутнє агентів, зокрема Klarna, Replit, Elastic та інші.

LangGraph є дуже низькорівневим і повністю зосереджений на оркестрації агентів .

Стабільні основні API

Примітиви графа (стан, вузли, ребра) та модель виконання/виконання залишаються незмінними, що спрощує оновлення.

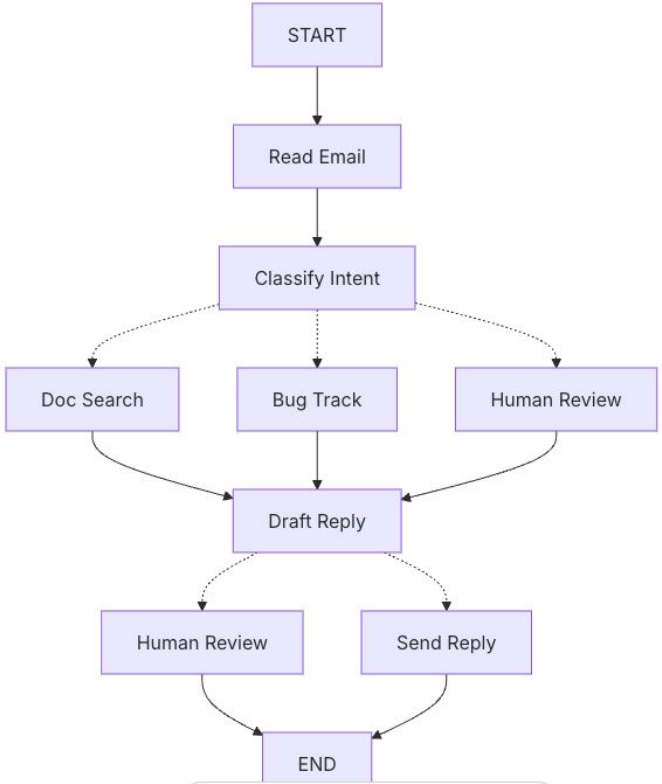
Надійність, за замовчуванням

Надійне виконання з контрольними точками, персистентністю, потоковою передачею та взаємодією з людиною залишається першокласним.

Безперебійно працює з LangChain v1

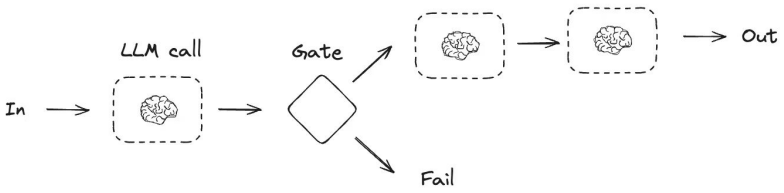
LangChain `create_agent` працює на LangGraph. Використовуйте LangChain для швидкого запуску; перейдіть до LangGraph для власної оркестрації.

Планування

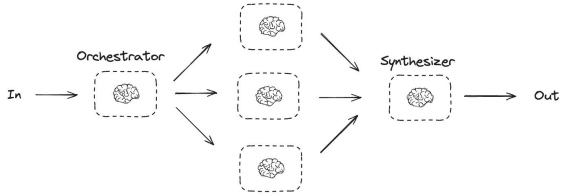

START
Read Email
Classify Intent
Doc Search
Bug Track
Human Review
Draft Reply
Human Review
Send Reply
END

Workflows

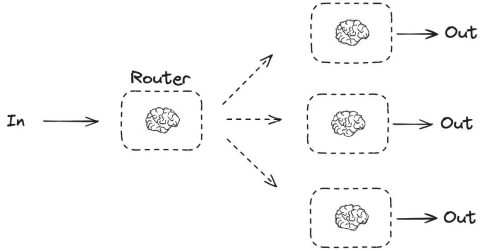
ланцюжок викликів



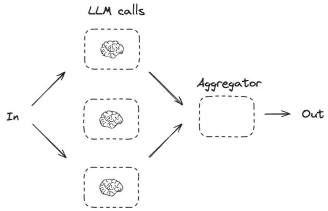
оркестратор



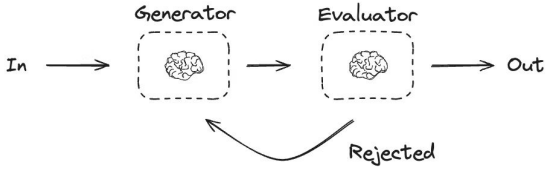
маршрутизація



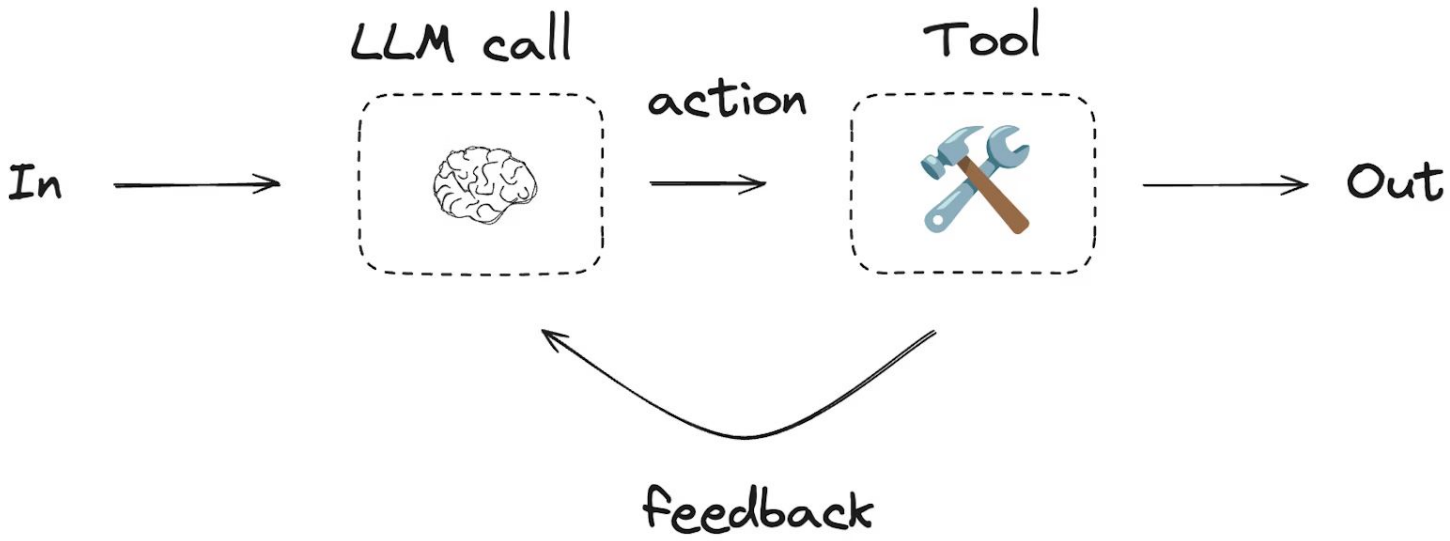
паралелізація



валідатор



Агенти



Практика

https://github.com/agentspro/module5